@llm

Appears in: **DL-02, DL-04**

shared/llm.py:8-17, 20-33

> *"Both patterns use the same LLM client and the same API layer, so they are interchangeable at the run level (@llm · @api_client)."*

@llm

shared/llm.py:8-17

```python
OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.1:8b")


def get_llm(temperature: float = 0.1) -> ChatOllama:
    return ChatOllama(
        model=OLLAMA_MODEL,
        base_url=OLLAMA_BASE_URL,
        temperature=temperature,
    )
```

...

shared/llm.py:20-33

```python
def call_json(llm: ChatOllama, system: str, user: str) -> dict:
    """Call LLM, parse JSON from response. Returns {fallback: True} on any failure."""
    try:
        response = llm.invoke([
            SystemMessage(content=system),
            HumanMessage(content=user),
        ])
        text = response.content.strip()
        # Strip markdown fences if present
        text = re.sub(r"^```(?:json)?\s*", "", text, flags=re.MULTILINE)
        text = re.sub(r"\s*```$", "", text, flags=re.MULTILINE)
        return json.loads(text.strip())
    except Exception as exc:
        return {"fallback": True, "error": str(exc)}
```

What it shows: The single shared Ollama client both Pipeline and Orchestra call. `call_json()` returns `{fallback: True}` on any parse failure instead of raising.

Source: shared/llm.py:8-17, 20-33 · image rendered by build_snippets.py